

Demonstration 1

Input Table:client id	client name	client type	client address

1 | John Doe | Individual | 123 Main St, Anytown, USA
2 | Jane Smith | Company | 456 Elm St, Anytown, USA
3 | Jim Beam | Company | 789 Oak St, Anytown, USA
4 | John Doe | Individual | 123 Main St, Anytown, USA
5 | Jane Smith | Company | 456 Elm St, Anytown, USA

Data Cleaning Operator Type: [op_replace_tag]

Output:

```
```python
rename the column "client id" to "cid"
input_table.rename(columns={"client type": "leixing", "client address":
"dizhi"}, inplace=True)
Print the renamed table
target_df = input_table
```
```

After executing the python code to make the table contain dirty data, we get a target_df:

client id | client name | leixing | dizhi
---|---|---|---

1 | John Doe | Individual | 123 Main St, Anytown, USA
2 | Jane Smith | Company | 456 Elm St, Anytown, USA
3 | Jim Beam | Company | 789 Oak St, Anytown, USA
4 | John Doe | Individual | 123 Main St, Anytown, USA
5 | Jane Smith | Company | 456 Elm St, Anytown, USA

We can restore the original table from the target_df by executing the Rename operator: **Rename(table_name="input_table", rename_map=[{"old_name": "leixing", "new_name": "client type"}, {"old_name": "dizhi", "new_name": "client address"}])**